

SQL in the Cloud

A reading guide · Warehouses, BigQuery and managed databases

So far you have run SQL against a MySQL database on your own laptop. In most jobs the data lives somewhere else: on a platform in the cloud that another team looks after. This guide explains the ideas you need to walk into that situation without being lost. It is for reading. You do not need a cloud account, and nothing here has to be installed.

The short version: **the SQL you have learned carries over almost unchanged. What changes is where it runs, who looks after it, and what a query costs.**

Part	Idea	The question it answers
1	Two kinds of data work	Why is recording an order different from analysing orders?
2	Why a data warehouse exists	Why is analysis done in a separate system?
3	Who looks after the database	What does a cloud provider do, and what stays with you?
4	BigQuery, a cloud warehouse	How is data organised and named there?
5	The same SQL in a new place	What carries over from MySQL, and what does not?
6	What a query costs	Why can ten rows be expensive?
7	Managed relational databases	Where does the MySQL you know live in the cloud?
8	Reading a result responsibly	What does a query result prove, and not prove?

How to read this guide

- Each part explains one idea, gives an example, and ends with a **Why it matters** box.
- Code marked **MYSQL** runs on the `sales_db` database from class, and the results shown are real output.
- Code marked **BIGQUERY** is for reading. It is written for Google's BigQuery and needs a BigQuery account to run, so no results are shown for it.

Part 1 Two kinds of data work

Think of the shop behind our sales dataset on an ordinary day. Two very different things happen with its data.

- **Right now, the application.** A customer places an order. The system must record it at once: one new row in `orders` and a few in `order_items`. Each event touches very little data, but there are many of them every minute and none may be lost.
- **Later, the analyst.** Someone asks which region places the most orders. The answer is four rows, but producing it means reading every order.

You have already written both kinds of statement. The first looks up one order:

MYSQL

```
SELECT order_id, order_date, ship_mode, city
FROM orders
WHERE order_id = 'CA-2016-152156';
```

RESULT

order_id	order_date	ship_mode	city
CA-2016-152156	2016-11-08	Second Class	Henderson

MySQL finds that row through the primary key without looking at the other 1,979 orders. The second summarises all of them:

MYSQL

```
SELECT region, COUNT(*) AS orders
FROM orders
GROUP BY region
ORDER BY orders DESC;
```

RESULT

region	orders
West	609
East	559
Central	478
South	334

Four rows come back, but all 1,980 orders were read to produce them. The pattern of work a system must handle is called its **workload**, and these two have names.

	OLTP: Online Transaction Processing	OLAP: Online Analytical Processing
Purpose	Run the business	Understand the business
Typical action	Insert, update or fetch a few rows	Scan, group and summarise many rows
Data it cares about	The current state: this order, this customer	History: months and years of orders
Typical SQL	A targeted lookup or a write	Aggregates, GROUP BY, rankings, trends
What must be fast	Many small operations, reliably	A few very large scans

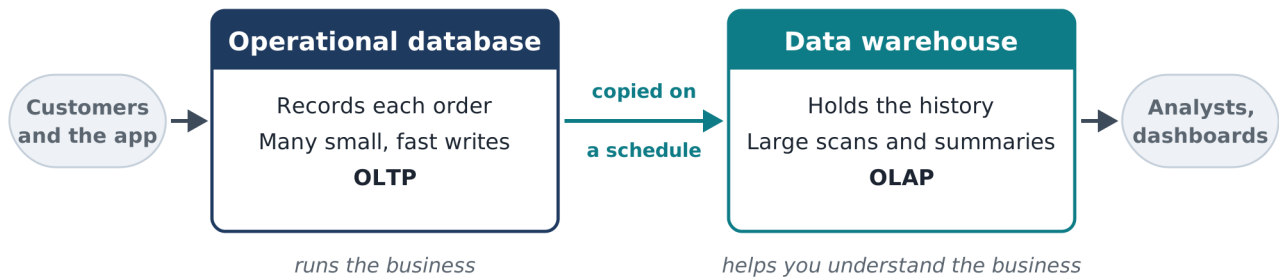
Watch out: OLTP versus OLAP is not small data versus big data. A transactional system can hold years of history, and an analytical dataset can be small. What differs is the kind of work, not the size of the table.

Why it matters: As a data analyst or data scientist, almost every query you write is analytical. Knowing that tells you which system you should be querying, and why you are rarely given access to the live application database. When you meet a new data platform, the first question to ask is which workload it was built for.

Part 2 Why a data warehouse exists

If the orders are already in the application's database, why not analyse them there? A small business often does. The trouble starts as it grows. A large analytical query reads millions of rows, and if it runs on the same machine as the checkout, the two compete. A slow report should never be the reason a customer's payment page hangs.

So organisations keep two systems. The **operational database** serves the application. The **data warehouse** is a separate system built for reading and summarising large amounts of history, and the data is copied into it on a schedule.



That copying step is ETL. The Day 2 workbook builds a small version of it in Part 11, where a `customer_summary` table is filled from the detailed tables.

Rows and columns on disk

The two systems also store data differently. An operational database stores a table row by row, which suits fetching or changing one order. A warehouse usually stores it column by column. An analytical question typically needs a few columns from a very large number of rows, and a column store reads only those columns.

Stored row by row

Fetch order A-103: read one row

order_id	region	product	quantity	sales
A-101	West	Chair	2	310.00
A-102	East	Paper	5	22.40
A-103	South	Phone	1	649.99
A-104	West	Binder	3	18.75
A-105	East	Desk	1	480.00

*Everything about one order sits together.
Suits an operational database.*

Stored column by column

Total sales by region: read two columns

order_id	region	product	quantity	sales
A-101	West	Chair	2	310.00
A-102	East	Paper	5	22.40
A-103	South	Phone	1	649.99
A-104	West	Binder	3	18.75
A-105	East	Desk	1	480.00

*The other three columns are never touched.
Suits a warehouse.*

Our own spreadsheet shows the scale of the saving. `sales_raw` has 21 columns. Total sales by region needs 2 of them, so a column store leaves the other 19 untouched.

Why it matters: Column storage is the reason behind the most important cost habit in Part 6: ask only for the columns you need. It also explains why nobody expects the warehouse to run the checkout, or the checkout database to answer five-year trend questions. Most organisations run both, and the skill is knowing which one a question belongs to.

Part 3 Who looks after the database

Your MySQL server is **self-managed**. You installed it and you start it, and if your laptop fails the data is gone unless you made a backup. In a company someone has to do that work for every database: install it, update it, back it up, watch it, and make it bigger when it fills up. Cloud providers offer to take over part of that work. How much they take over is the main difference between services.

Model	The provider does	You do	Example
Self-managed	Nothing	Everything: the server, installation, updates, backups, capacity, recovery	MySQL on your laptop or a company's own server
Managed database	Sets up the server, updates the software, takes backups, recovers from failure	Choose the engine, the size, the network and who may connect	Amazon RDS (Part 7)
Serverless platform	All of the above, and gives each query the computing power it needs	Bring the data and the queries, control access, watch the bill	BigQuery (Part 4)

Watch out: Managed does not mean nothing to manage. A badly written query does not become a good one because someone else runs the server. Nor is the cloud automatically cheaper or faster: it moves work and cost around, and some of both stays with you.

What always stays with your organisation

- **Location.** Cloud data is stored in a chosen region of the world. Privacy laws such as POPIA in South Africa and GDPR in Europe limit where personal information may be sent, so the location of a dataset is a decision, not a detail.
- **Access.** Someone must decide who can read and change what. Cloud platforms call this IAM (identity and access management). It is the same idea as `CREATE USER` and `GRANT` on Day 1: give each person only what the job needs.
- **Cost and governance.** Budgets, limits and review of what is being run.

Why it matters: This is where roles divide. In a large company, data engineers and platform teams usually do the provider's and the administrator's share, and you may never set up a server. In a small company you may be the whole data team. In both, four things stay with whoever writes the query: using the right system, respecting access rules, the cost of the query, and what the result is taken to mean.

Part 4 BigQuery, a cloud warehouse

BigQuery is Google Cloud's data warehouse. It is fully managed and **serverless**. Servers still exist in Google's data centres, but you never choose, size or start one. You sign in on a web page, write SQL and run it.

How data is organised

BigQuery arranges data in three levels. They map closely onto what you already know.

BigQuery	Closest thing in our MySQL setup	What it is
Project	Your MySQL server	The boundary for permissions, billing and the queries you run
Dataset	A database, such as <code>sales_db</code>	A container for tables. It has a location.
Table	A table, such as <code>orders</code>	Rows and columns with a defined schema

A table's full name gives all three levels, separated by dots and wrapped in backticks:

BIGQUERY · [for reading](#)

```
SELECT start_time, start_station_name, subscriber_type, duration_minutes
FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`
LIMIT 20;
```

Here `bigquery-public-data` is the project, `austin_bikeshare` is the dataset and `bikeshare_trips` is the table: one row for every bike-share trip taken in Austin, Texas. MySQL has a two-level version of the same thing: `sales_db.orders`.

Three ideas that are new

- **A query runs as a job.** You submit SQL, the platform runs it and returns the result together with statistics about the work done: how much data was read and how long it took.
- **Storage and computing power are separate.** The data sits in storage. Computing power is lent to each query for as long as it runs. You do not size a server in advance, and the cost follows the data a query processes.
- **Public datasets.** Google hosts shared datasets that anyone can query, including the bike-share trips above. Google pays to store them. Whoever runs a query is responsible for the cost of that query.

Look before you query

On your laptop you looked at a new table with `DESCRIBE` and `SELECT * ... LIMIT 5`. BigQuery offers both without running a query. The **Schema** tab lists every column and its type, and the **Preview** tab shows sample rows. Both are free. Part 6 explains why `SELECT *` is the wrong tool for this in a warehouse.

Why it matters: BigQuery is one of several cloud warehouses. Snowflake, Amazon Redshift, Microsoft Fabric and Databricks are others, and which one you meet depends on where you work. Nobody learns them all in advance. They share the ideas in this part: containers inside containers, queries that run as measured jobs, and cost tied to the work done. Learn the ideas once and each new platform is mostly a matter of new names.

Part 5 The same SQL in a new place

Here is a question about the bike-share data: how many trips does each type of rider take, and how long is the average trip?

BIGQUERY · [for reading](#)

```
SELECT subscriber_type,
       COUNT(*)           AS trips,
       AVG(duration_minutes) AS avg_duration_minutes
FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`
GROUP BY subscriber_type
ORDER BY trips DESC;
```

And here is a question about our own data: how many orders use each ship mode, and how many days does shipping take on average?

MYSQL

```
SELECT ship_mode,
       COUNT(*)           AS orders,
       AVG(DATEDIFF(ship_date, order_date)) AS avg_days_to_ship
FROM orders
GROUP BY ship_mode
ORDER BY orders DESC;
```

RESULT

ship_mode	orders	avg_days_to_ship
Standard Class	1171	5.0256
Second Class	393	3.2392
First Class	318	2.1698
Same Day	98	0.0408

Line for line it is the same query: a grouping column, a count, an average, **GROUP BY** and **ORDER BY**. **SELECT**, **WHERE**, **JOIN**, **LIMIT** and CTEs all work the same way too.

What does differ: the dialect

Every database system speaks its own **dialect** of SQL. BigQuery's is called GoogleSQL. The differences are in function names and small details such as these:

To do this	MySQL	BigQuery
Name a table	<code>sales_db.orders</code>	<code>`project.dataset.table`</code>
Days between two dates	<code>DATEDIFF(ship_date, order_date)</code>	<code>DATE_DIFF(ship_date, order_date, DAY)</code>
Year of a date	<code>YEAR(order_date)</code>	<code>EXTRACT(YEAR FROM order_date)</code>

The documentation or a quick search settles differences like these in a minute. The reasoning behind the query does not change.

A trap that travels with you: time zones

The `start_time` column is a timestamp, and BigQuery reads timestamps in UTC unless told otherwise. "At what hour of the day do riders in Austin start their trips?" is a question about Austin's local time, so the query has to say so:

BIGQUERY · for reading

```
SELECT EXTRACT(HOUR FROM start_time AT TIME ZONE 'America/Chicago') AS start_hour,
       COUNT(*) AS trips
FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`
GROUP BY start_hour
ORDER BY start_hour;
```

Without `AT TIME ZONE` the query still runs and still returns a tidy table of hours. They are simply shifted by five or six hours, so the morning rush appears in the early afternoon. The same trap waits for a South African business: 08:00 in Johannesburg is 06:00 UTC.

Why it matters: The most valuable thing you have learned is the reasoning, and it is portable to any SQL platform. Two things do not travel automatically: function names, and assumptions about the data such as its time zone. A query that runs without an error is not necessarily answering the question that was asked.

Part 6 What a query costs

On your laptop a query costs nothing you can see: the database is small and already running. A cloud warehouse measures every query. Under BigQuery's standard pricing you pay for the amount of data a query reads. At the time of writing (October 2026) the list price is \$6.25 per TiB read (a TiB is a little over 1,000 GB), and the first TiB each month is free.

Rows returned and bytes processed are different numbers

Rows returned describes the output. **Bytes processed** describes how much stored data the platform had to read to produce it. Compare two queries on the bike-share table:

BIGQUERY · for reading

```
-- Query A: four named columns
SELECT start_time, start_station_name, subscriber_type, duration_minutes
FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`
LIMIT 10;

-- Query B: every column
SELECT *
FROM `bigquery-public-data.austin_bikeshare.bikeshare_trips`
LIMIT 10;
```

Both return at most 10 rows. Query A reads four columns of the table. Query B reads all of them. Two habits from the laptop have to be unlearned:

- **SELECT *** is a poor default. The table is stored column by column (Part 2), so asking for every column reads every column, whether the question needs it or not.
- **LIMIT** is not a cost control. It limits how many rows come back. On an ordinary table it does not reduce how much data is read to find them.

Putting a price on it

The numbers in this example are made up to show the arithmetic. Picture a company table of website events: 2 TiB in total, across 50 columns of similar size.

Query	Data read	Cost at \$6.25 per TiB
SELECT * ... LIMIT 10	All 50 columns: 2 TiB	\$12.50, for ten rows
SELECT of the 3 columns needed	3 of 50 columns: 0.12 TiB	About \$0.75
The first query behind a dashboard that refreshes every hour	24 runs a day: 48 TiB	\$300 a day

See the cost before you run

BigQuery's editor shows an estimate of the data a query will process before you press Run. Reading that number first is the professional habit, and it fits into five steps:

1. **Preview** the table. Do not run `SELECT *` to look at it.
2. **Name the columns** the question needs.
3. **Check the estimate.**
4. **Decide** whether that amount is reasonable for the question.
5. **Run**, then interpret the result.

Partitions: reading less of a big table

Very large tables are usually split into **partitions**, most often by date: one partition per day or per month. When a query filters on that date column, the platform skips every partition outside the range. This is called **partition pruning**.

No filter on the date



All 12 monthly partitions are read.

Filter: trip_date in September



One partition is read; the other 11 are skipped.

The table name below is invented for the example. If `trips` is partitioned by `trip_date`, this query reads September and nothing else:

BIGQUERY · for reading

```
SELECT COUNT(*) AS trips
FROM `my-project.rides.trips`
WHERE trip_date BETWEEN DATE '2026-09-01' AND DATE '2026-09-30';
```

Clustering is a related idea: the rows inside a table are kept sorted by chosen columns, so a filter on those columns also reads less. You will rarely set up either yourself. You will be expected to use them, which means knowing which column a table is partitioned on and filtering on it.

BigQuery also has a second pricing model, in which a company pays for reserved computing power instead of per query. The habits above matter under both, because a wasteful query then takes capacity away from everyone else.

Why it matters: This is the part of the cloud that a data engineer cannot do for you. They build the tables. You write the queries, and the bill or the slow shared system follows the query. A new analyst's first expensive mistake is nearly always an unfiltered `SELECT *` on the largest table. The habits that prevent it cost nothing: name your columns, filter on the date, read the estimate.

Part 7 Managed relational databases

BigQuery is the analytical side. The operational side has cloud services too, and they are much closer to what you have been using.

- **Amazon RDS** (Relational Database Service) runs familiar database engines for you, including MySQL and PostgreSQL. Amazon sets up the server, updates the software, takes backups and recovers from failures. You still choose the engine and version, the size of the machine, the region, who can connect and how long backups are kept.
- **Amazon Aurora** is part of the RDS family: Amazon's own engine, built to be compatible with MySQL and PostgreSQL. It runs as a cluster, with several database servers sharing one managed storage layer.

The MySQL you have learned is exactly what runs there. Workbench would connect to an address such as `mydb.abc123.af-south-1.rds.amazonaws.com` in place of `localhost`, and the statements from Days 1 and 2 would work unchanged.

	BigQuery	RDS and Aurora
Kind of service	Serverless data warehouse	Managed relational database
Built for	Large analytical queries (OLAP)	An application's transactions (OLTP)
Decided before the first query	Almost nothing	Engine, machine size, storage, network, backups
What you keep an eye on	Data processed by each query	The size and health of the server
How you reach your data	Project, dataset, table	A server address, then database and table

Neither replaces the other, and many organisations use both: Aurora or RDS behind the application, a warehouse for analysis.

The same ideas under other names

Each cloud provider sells its own version of both kinds of system. The names differ and the ideas in this guide apply to all of them.

Kind of system	Google Cloud	Amazon (AWS)	Microsoft Azure	Independent
Data warehouse	BigQuery	Redshift	Fabric, Synapse	Snowflake, Databricks
Managed relational database	Cloud SQL	RDS, Aurora	Azure SQL Database	

Why it matters: If you join a team that builds an application, its data will very likely sit in a managed relational database like these, and your MySQL carries straight over. And when someone names a service you have never met, two questions place it: is it built for transactions or for analysis, and how much of the running is done for us?

Part 8 Reading a result responsibly

The last idea has nothing to do with the cloud and applies to every query you will ever run. For any result, ask three questions: what does it mean, what does it not prove, and what else would I want to know? Take sales and profit margin by category in our own data:

MYSQL

```
SELECT p.category,
       SUM(oi.sales) AS total_sales,
       ROUND(SUM(oi.profit) / SUM(oi.sales) * 100, 1) AS margin_pct
FROM order_items AS oi
JOIN products AS p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_sales DESC;
```

RESULT

category	total_sales	margin_pct
Technology	347778.0740	12.8
Furniture	293001.2040	2.1
Office Supplies	282523.8750	16.0

Question	For this result
What it means	In these four years of orders, furniture kept about 2 cents of profit from each dollar of sales, against 16 cents for office supplies.
What it does not prove	Why the furniture margin is low, or that the business should stop selling furniture. The query describes a difference. It does not explain it.
What else you would want	Discount levels in each category, whether a few large loss-making orders drive the figure, and whether furniture buyers also buy the profitable products.

Why it matters: No platform does this for you. A query can be cheap, fast and correctly written, and its result can still be read as saying more than it does. Stating plainly what a result does not show is a large part of being an analyst whose numbers people rely on.

Putting it together

When you meet any new data platform, five questions in this order will orient you:

1. **What workload is it built for?** Transactions or analysis (Parts 1 and 2).
2. **Who runs what?** What the provider does and what stays with us (Part 3).
3. **How is the data organised and named?** (Part 4)
4. **What does a query cost, and how do I see that before I run it?** (Part 6)
5. **What does my result mean, and what does it not prove?** (Part 8)

OPTIONAL: TRY IT AT HOME

BigQuery has a free Sandbox that needs no credit card. It allows 1 TiB of queries a month, far more than the queries in this guide need. It does not allow statements that change data (`INSERT`, `UPDATE`, `DELETE`). Follow the S06 Cloud Setup Guide that comes with the course slides to open it, find the bike-share table, and read the estimate under the editor before running each BigQuery query from Parts 4 to 6.

Key terms

Term	Meaning
Workload	The pattern of work a system must handle: what kind of operations, how often, on how much data.
OLTP	Online Transaction Processing. Many small, fast, reliable operations that run the business.
OLAP	Online Analytical Processing. Queries that scan and summarise large amounts of history.
Data warehouse	A separate system built for analytical queries, filled with data copied from operational systems.
Columnar storage	Storing a table column by column, so a query reads only the columns it names.
Managed service	A service where the provider does routine administration such as updates and backups.
Serverless	A service where you never choose or size a server; the platform supplies computing power per query.
Project, dataset, table	BigQuery's three levels for organising data.
Query job	One run of a query in BigQuery, with statistics about the work it did.
Bytes processed	The amount of stored data a query reads. It drives cost, and differs from rows returned.

Term	Meaning
Partition	A slice of a large table, usually one per day or month. A date filter lets the platform skip the rest.
Clustering	Keeping a table's rows sorted by chosen columns so filters on them read less data.
Dialect	One system's version of SQL. BigQuery's is called GoogleSQL.
IAM	Identity and access management: who is allowed to see and do what.

Check yourself

1. An app updates one customer's payment status. Is that operational or analytical work, and why?
2. In `bigquery-public-data.austin_bikeshare.bikeshare_trips`, which part is the dataset?
3. Two queries each return 10 rows. Must they process the same amount of data?
4. Why does the hour-of-day query in Part 5 use `AT TIME ZONE 'America/Chicago'`?
5. One bike station has the highest trip count. What can you safely say, and what can you not?
6. Name two decisions RDS asks you to make that BigQuery does not.

Answers

1. Operational (OLTP). It changes one row, must happen immediately and must not be lost.
2. `austin_bikeshare`. The project comes first and the table last.
3. No. Rows returned and bytes processed are different numbers. A `SELECT *` reads every column even when `LIMIT 10` keeps the output small.
4. The question is about local time in Austin, and timestamps are read in UTC by default. Without it the hours are shifted and the answer is wrong for the question.
5. You can say it appears most often as a starting point in this data. You cannot say it is the best station, why it is busy, or that it needs more bikes; that needs other data.
6. Any two of: the database engine and version, the size of the machine, storage, the network and who can connect, how long backups are kept.